

Universidad Nacional de Cuyo

Facultad de Ingeniería

Lic. en Ciencias de la Computación

Licenciatura en Ciencias de la Computación

Compiladores
Manual de Usuario de TinyRust+

Rocío Belén Corral

Universidad Nacional de Cuyo

Facultad de Ingeniería

Mayo 2023

Introducción **3**
Primeros pasos **3**
Instalación 3
Compilación 3
Ejecución 4
Errores 5
Estructura general **7**
Comentarios 7
Variables 7
Identificadores 8
Palabras reservadas 8
Tipos y clases predefinidas 9
I32 9
Bool 9
Str 10
Char 12
Array 12
Object 14
IO 14
Literales nulos 15
Clases 17
Atributos 17
Métodos 18
Constructor 20
Herencia 21
Tipos 22
Sentencias **23**
Asignación 23
Sentencia simple 24
Bloque 24
Condicional 24
Bucle 25
Retorno de método 26
Expresiones **27**
Operaciones 27
Llamada a métodos 27
Acceso a atributos 28
New 29
Self 30
Anexo 1: alfabeto de entrada **31**
Anexo 2: gramática EBNF **32**

Introducción

TinyRust+ es un lenguaje de programación multipropósito orientado a objetos, type safe y con sintaxis similar al lenguaje Rust. Es un lenguaje reducido, pero igualmente incluye características propias de los lenguajes de programación modernos. TinyRust+ es compilado y fuertemente tipado, por lo que impone una disciplina fuerte de programación y garantiza que no ocurran errores de tipo en tiempo de ejecución.

Primeros pasos

Un código fuente a compilar en TinyRust+ consiste de un único archivo, ya que el lenguaje no posee un sistema de importado/exportado de dependencias. El archivo debe tener extensión *.rs*. El siguiente fragmento de código muestra el ejemplo más básico de código en TinyRust+.

```
// holaMundo.rs

fn main() {
  (IO.out_str("Hola Mundo!"));
}
```

Este ejemplo muestra por pantalla la cadena “Hola mundo!” y sirve como un primer ejemplo para probar el correcto funcionamiento del lenguaje.

Vale la pena aclarar que aunque la extensión *.rs* coincide con la del lenguaje Rust, los archivos de TinyRust+ no tienen exactamente la misma sintaxis. Por lo que obtendrá un error si se intenta compilarlos con un compilador de Rust.

Instalación

El compilador de TinyRust+ consiste en un archivo portable en formato *JAR*[1], por lo que requiere tener instalado Java para su funcionamiento. La versión de Java recomendada es la 17. En versiones previas no se garantiza el correcto funcionamiento del compilador.

Compilación

Tras descargar el compilador de TinyRust+, un programa en este lenguaje se compila invocando al compilador y especificando la ruta del archivo de entrada y, opcionalmente, la ruta del archivo de salida.

```
java -jar <ruta del compilador>.jar <ruta archivo entrada>.rs
[<ruta archivo salida]
```

Por ejemplo, para compilar el archivo de ejemplo previo se podría ejecutar el siguiente

[1] Java Archive (JAR) Files

comando:

```
java -jar ~/tinyRust.jar ./holaMundo.rs ./out/holaMundo
```

Si no se especificara un destino para el archivo de salida, este será generado por defecto en la misma ubicación que el archivo de entrada y con el mismo nombre, pero con extensión *ASM*[2].

El archivo de salida posee código assembler MIPS32[3], por lo cual es ejecutable dentro de un simulador de esta arquitectura, como SPIM, MARS o similares.

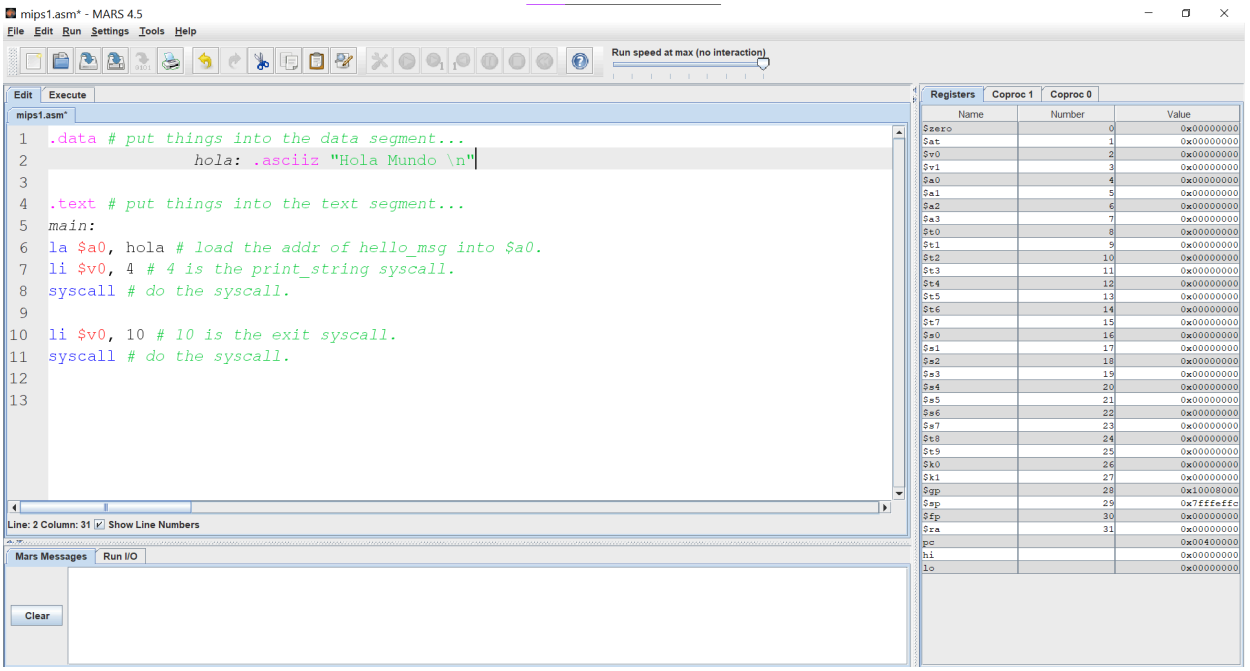
Para el caso del archivo de ejemplo, la salida por pantalla esperada sería la siguiente:

```
Hola Mundo!
```

Ejecución

En este manual se explica cómo utilizar el simulador MARS versión 4.5 para ejecutar los archivos compilados de TinyRust+, pero también es posible utilizar cualquier otro simulador de la misma arquitectura.

El simulador MARS se puede encontrar en formato JAR ejecutable. Una vez que el programa se ha descargado y abierto, posee una interfaz como la siguiente:

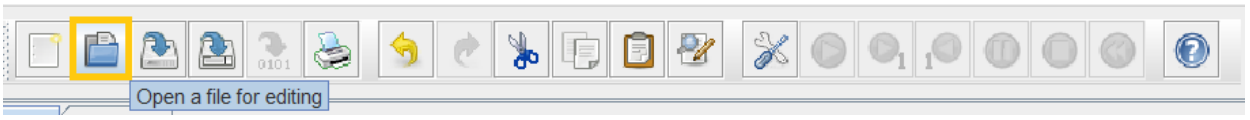


La interfaz posee una barra de acciones en la parte superior, en el panel principal se encuentra el código ASM para ejecutar y en el panel inferior se muestra la salida del ensamblador y la entrada/salida durante la ejecución.

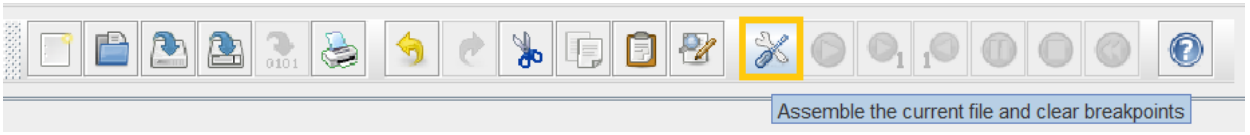
Para comenzar, se debe cargar en el editor el archivo ASM generado por el compilador de

[2] Formato de archivo ASM - Formato de archivo de código fuente de lenguaje ensamblador
[3] Imagination's MIPS32 architecture

TinyRust+. Esto se puede lograr eligiendo la acción “Abrir un archivo” en la barra superior y luego seleccionando el archivo en el selector de archivos emergente.



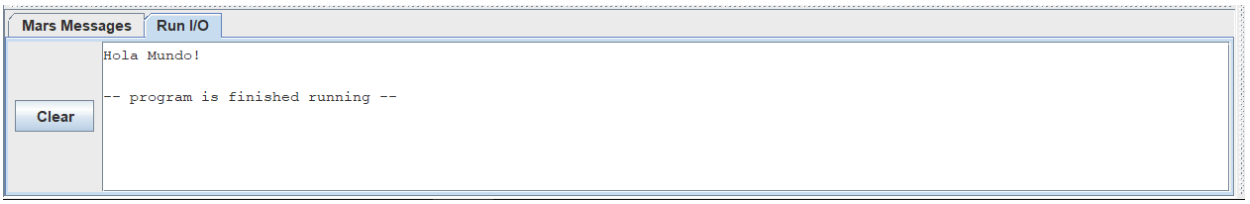
Luego de cargar el archivo, su código se puede ver en el panel principal. Este código es el equivalente semántico en MIPS32 del código de TinyRust+ original. El siguiente paso es ensamblarlo. Esta acción también se realiza desde el menú superior, eligiendo la acción “Ensamblar archivo actual”.



Cuando el código ha sido ensamblado, la aplicación se mueve a la pestaña de ejecución. En esta pestaña finalmente se puede ejecutar el código eligiendo la acción “Ejecutar el programa actual” en el menú superior.



Durante la ejecución, la entrada y salida se verá en el panel inferior de la pantalla, donde se puede leer y escribir siguiendo el flujo del programa original. Para el archivo de ejemplo, la salida es la siguiente:



Errores

En caso de que ocurriera un error durante la compilación del archivo de entrada, no se generará el archivo de salida y en cambio se mostrará por pantalla la información del error: línea y columna donde ocurrió, tipo del error e información explicativa.

Por ejemplo, si se compilara el mismo archivo de ejemplo *holaMundo.rs*, pero quitando la llave de cierre del bloque en la última línea.

```
// holaMundo.rs

fn main() {
  (IO.out_str("Hola mundo!"));
```

```
// Falta la llave de cierre!
```

Se obtendría un error sintáctico como el siguiente:

```
ERROR: ANALISIS SINTACTICO
| NUMERO DE LINEA: | NUMERO DE COLUMNA: | DESCRIPCION: |
| LINEA 4 | COLUMNA 1 | TOKEN INESPERADO: SE ESPERABA EN ORDEN O UNA
DECLARACION DE VARIABLE O SENTENCIA O "}" Y SE ENCONTRO "End Of File"
|
```

TinyRust+ busca ser un lenguaje seguro, por lo cual la mayoría de los errores posibles se detectan en tiempo de compilación. En el proceso de compilación se detectan errores léxicos (generalmente al usar caracteres ilegales o cometer *typos*), errores sintácticos (estructura inválida del programa) y errores semánticos (declaraciones duplicadas, operaciones inválidas de tipos, herencia cíclica, uso de tipos no declarados). Luego en tiempo de ejecución, son pocos los errores que pueden ocurrir, los cuales tienen que ver principalmente con el valor de expresiones resueltas durante la ejecución, como pueden ser división por cero y accesos fuera de índice en arreglos.

Estructura general

Todo programa de TinyRust+ está compuesto primero de la declaración de cero o más clases y luego la declaración de la función *main* obligatoriamente al final del archivo.A la hora de la ejecución, el punto de entrada del programa es el método *main*. Es decir, únicamente se ejecutará el código dentro del método *main* y, en consecuencia, sólo se ejecutarán los constructores y métodos de las clases declaradas que se invoquen dentro del bloque de *main* o como consecuencia de estas invocaciones.

⟨program⟩ ::= ⟨Clase⟩* ⟨Método-Main⟩

⟨Método-Main⟩ ::= **`fn main ()`** ⟨Bloque-Método⟩

Dentro del método *main* se pueden declarar variables de tipos predefinidos o clases definidas en el mismo archivo seguidas por sentencias que utilicen estas variables, como bucles, condicionales, asignaciones e invocaciones a métodos. En las siguientes secciones de esta guía se explica el funcionamiento de cada uno de estos elementos.

Comentarios

Los comentarios son fragmentos de código que son ignorados por el compilador, pero pueden resultar de ayuda para el programador. TinyRust+ tiene dos tipos de comentarios:

- `/* comentario multilinea */`
  En este tipo de comentario todos los caracteres desde la apertura `/*` hasta el cierre `*/` son ignorados. Iniciar un comentario multilínea y no cerrarlo generará un error en tiempo de compilación.
- `// Comentario simple`
  Todos los caracteres desde `//` hasta el final de la línea son ignorados.

Dentro de un comentario se pueden utilizar todos los tipos de caracteres válidos del lenguaje (detallados en el Anexo 1).

Variables

Las variables locales en TinyRust+ deben declararse explícitamente antes de usarse y no se pueden declarar y asignar en la misma línea. Dado que el lenguaje es estáticamente tipado y no implementa inferencia de tipos, se requiere que se especifique el tipo de la variable. Es posible declarar múltiples variables del mismo tipo en una sola línea. La sintaxis de declaración de una variable es la siguiente:

⟨Decl-Var-Locales⟩ ::= ⟨Tipo⟩ **`:`** ⟨Lista-Declaración-Variables⟩ **`;`**

⟨Lista-Declaración-Variables⟩ ::= **`id`** | **`id ,`** ⟨Lista-Declaración-Variables⟩

El alcance de las variables declaradas es léxico, es decir, son accesibles dentro del bloque en que son declaradas y en sub-bloques anidados de este.

Identificadores

Un identificador es una cadena que puede estar compuesta por cualquier combinación de letras mayúsculas y minúsculas, dígitos y guiones bajos. Los identificadores se utilizan como nombres de variables y clases.

Para una variable el identificador debe comenzar con una letra minúscula. La única restricción extra es que el identificador no puede coincidir con ninguna palabra reservada del lenguaje ni ser “self” o “void”.

Los identificadores *self* y *void* están declarados implícitamente y son tratadas de manera especial dentro del lenguaje, como se explicará en siguientes secciones.

Tanto los atributos de clases, argumentos de métodos, como variables locales son distintos tipos de variables. El alcance de los atributos se extiende a todos los métodos de la clase, mientras que el alcance de los argumentos y variables locales está restringido al método de su declaración. La vinculación de una referencia de identificador es el ámbito más interno que contiene una declaración para ese identificador. Es decir, si no se halla una declaración de variable local o argumento con ese nombre, la referencia se vincula al atributo del mismo nombre. La excepción a esta regla es el identificador *self*, que está implícitamente vinculado en cada clase. Por este motivo, el identificador *self* no es accesible en el método *main* (ya que no hay ninguna clase vinculada), pero no se puede utilizar como identificador de ninguna variable.

Palabras reservadas

Las palabras reservadas de TinyRust+ no se pueden utilizar como identificadores, ya que se usan para estructurar sentencias o como literales dentro del programa. Estas palabras son: *class*, *else*, *false*, *if*, *return*, *while*, *true, nil*, *false*, *new*, *fn*, *static*, *pub* y *self*. En TinyRust+ las palabras reservadas son sensibles a mayúsculas y minúsculas.

Tipos y clases predefinidas

TinyRust+ cuenta con un conjunto reducido de clases predefinidas que se pueden utilizar para declarar literales de ciertos tipos o acceder a un conjunto de métodos que pueden ser de utilidad para manipular este tipo de dato.

En este lenguaje no está permitido declarar clases cuyo nombre coincida con el de una clase predefinida, dado que en tal caso la clase predefinida quedaría inaccesible.

Las clases predefinidas son: I32, Bool, Str, Char, Array, Object e IO. Las primeras cinco de estas clases (I32, Bool, Char y Array) representan tipos de dato predefinidos de TinyRust+ y no se permite heredar de ninguna de estas clases. Mientras que Object e IO no están asociadas a ningún tipo de dato en particular y sí se puede heredar de ellas.

I32

La clase I32 representa números enteros de 32 bits. Es decir que se pueden guardar números enteros en el rango entre -2.147.483.648 y +2.147.483.647. I32 es un tipo de dato primitivo del lenguaje.

Esta clase no posee métodos especiales y el valor por defecto de una variable de este tipo es 0.

Los literales enteros son cadenas no vacías de dígitos del 0 al 9. Todo literal entero en TinyRust+ es de tipo I32 y se puede usar como expresión directamente, sin necesidad de declarar una variable de este tipo.

```
fn main() {
  I32: x; // Declarar variable.
  (IO.out_i32(x)); // Salida: 0

  x = 10; // Asignar variable.
  (IO.out_i32(x + 5)); // Salida: 15
}
```

Bool

La clase Bool representa valores booleanos. Sus dos valores posibles son true y false. Bool es un tipo de dato primitivo del lenguaje.

Esta clase no posee métodos especiales y el valor por defecto de una variable de este tipo es false.

Tanto true como false son literales booleanos y se pueden usar como expresión directamente, sin necesidad de declarar una variable de este tipo.

```
fn main() {
  Bool: activo; // Declarar variable.
  (IO.out_bool(activo)); // Salida: false
  (IO.out_bool(!activo)); // Salida: true

  activo = true; // Asignar variable.
  (IO.out_bool(activo || false)); // Salida: true
}
```

Str

La clase Str representa cadenas de caracteres de cualquier longitud. Los literales de esta clase se especifican entre comillas dobles y se pueden usar como expresión directamente, sin necesidad de declarar una variable de este tipo. El valor por defecto de una variable de este tipo es la cadena vacía “”. Str es un tipo de dato primitivo del lenguaje.

```
fn main() {
  Str: saludo, nombre; // Declarar variables.
  (IO.out_str("")); // La salida es una linea vacia.
  (IO.out_str("Hola mundo!")); // Salida: Hola mundo!

  saludo = "Hola ";
  nombre = IO.in_str(); // Input del usuario. Ejemplo: "dev".
  (IO.out_str(saludo.concat(nombre))); // Salida: Hola dev
}
```

Los caracteres que se admiten dentro de un literal cadena coinciden con el alfabeto de entrada de TinyRust+ exceptuando el salto de línea. Es decir, todos los símbolos válidos del lenguaje (excepto el salto de línea) pueden utilizarse dentro de un literal cadena. En el Anexo 1 se encuentran listados por extensión los caracteres aceptados por el lenguaje. Vale destacar que caracteres con tilde y la letra *ñ* no se encuentran incluídos en esta lista, por lo que el programa arrojará un error en tiempo de compilación al hallar un caracter con tilde o *ñ* en cualquier parte del código, ya sea dentro de un literal, en un comentario o en un identificador. En el caso de querer utilizar el caracter comillas dobles, se debe escapar con una barra invertida inmediatamente antes. Por ejemplo, el siguiente es un literal cadena válido para TinyRust+.

```
"La clave es \"root\". \n Usar comillas simples y el caracter
'@' es posible! Se pueden usar \\slashes y \ttabs tambien,
pero no se pueden usar tildes."
```

Además de las comillas dobles, también hay otro tipo de caracteres especiales que se pueden escapar.

- `\n` se puede usar para representar un salto de línea.
- `\t` se puede usar para representar una tabulación..
- `\\` sirve para usar el caracter barra invertida sin que signifique un escape.
- Para cualquier otro caracter x, `\x` representa el mismo caracter x.

El único caracter que no se permite escapar es 0. Dado que `\0` representa el caracter nulo, que no está incluido en el alfabeto de TinyRust+.

Para el ejemplo anterior, la salida por pantalla del literal en tiempo de ejecución sería la siguiente:

```
La clave es "root".
 Usar comillas simples y el caracter 'ñ' es posible! Se pueden
usar \slashes,     tabs y @arrobas tambien, pero no se puede
usar tildes."
```

La clase Str provee un conjunto de métodos que se pueden invocar en variables de este tipo:

- `fn length() -> I32.`
  length devuelve la longitud de la cadena, es decir, la cantidad de caracteres que contiene. Es un número entero no negativo.
- `fn concat(Str: s) -> Str`
  concat devuelve la cadena formada al concatenar *s* al final de la cadena contenida en la variable sobre la que se invoca. No modifica el valor original de la variable.
- `fn substr(I32: i, I32: l) -> Str`
  substr devuelve una subcadena de la cadena almacenada en la variable sobre la que se invoca, comenzando en la posición i con longitud l. Las posiciones de cadena se numeran comenzando en el índice 0. Tanto i como l deben ser no negativos. Se generará un error en tiempo de ejecución si la subcadena especificada está fuera de rango o si alguno de los dos argumentos es negativo.

Vale destacar que no se puede invocar estos métodos sobre literales directamente, sino que se deben invocar sobre variables declaradas de este tipo. Este comportamiento se ilustra en el siguiente fragmento de código.

```
fn main() {
  Str: saludo;
  saludo = "Hola mundo";

  (IO.out_i32(saludo.length())); // Salida: 10
  (IO.out_i32("Hola mundo".length())); // ERROR
```

```
  saludo = saludo.concat(" cruel"); // Salida: Hola mundo
cruel
  saludo = "Hola mundo".concat(" cruel"); // ERROR

  saludo = saludo.substr(0, 4); // Salida: Hola
  saludo = "Hola mundo cruel".substr(0, 4); // ERROR

}
```

Char

La clase Char representa cadenas de exactamente un caracter. Un literal de tipo caracter se especifica entre comillas simples y se puede usar como expresión directamente, sin necesidad de declarar una variable de este tipo. Esta clase no posee métodos especiales y el valor por defecto de una variable de este tipo es ' ' (espacio en blanco). Char es un tipo de dato primitivo del lenguaje.

```
fn main() {
  Char: caracter; // Declarar variable.
  (IO.out_char('h')); // Salida: h

  caracter = '\''; // Asignar variable.
  (IO.out_i32()); // Salida: '
}
```

El conjunto de caracteres aceptados para este literal es el mismo que para los literales de tipo Str explicados en la sección previa, con la única diferencia que se debe especificar un único caracter entre las comillas simples. En el caso de los caracteres escapados se considera un sólo caracter a la combinación de la barra invertida y el caracter a continuación. Por lo cuál la única excepción donde sintácticamente se aceptan dos caracteres dentro de las comillas simples es cuando el primer caracter es una barra invertida. Más allá de esta excepción, en caso de declararse un literal de este tipo conteniendo cero o más de un caracter dentro de las comillas simples, se generará un error en tiempo de compilación.

Array

La clase Array representa listas de tamaño estático de elementos de tipos primitivos (I32, Bool,Str, y Char). A la hora de declarar un arreglo, se debe especificar también cuál será el tipo de dato de sus elementos. Todos los elementos de un arreglo serán del mismo tipo primitivo. Dos arreglos son compatibles si el tipo de sus elementos es el mismo, por ejemplo, no se puede guardar una instancia de un arreglo de booleanos en una variable declarada de tipo Array con elementos de tipo I32.

A continuación se especifica la sintaxis de declaración, instanciación y acceso de un arreglo.

⟨Decl-Var-Array⟩ ::= **`Array`** ⟨Tipo-Primitivo⟩ **`:`** ⟨Lista-Declaración-Variables⟩ **`;`**

⟨Llamada-Constructor-Array⟩ ::= **`new`** ⟨Tipo-Primitivo⟩ **`[`** ⟨Literal-Entero⟩ **`]`**

⟨Acceso-Var-Array⟩ ::= **`id [`** ⟨Expresión⟩ **`]`**

El tamaño de un arreglo debe saberse en tiempo de compilación, por lo cual, al instanciarlo se debe especificar el tamaño del arreglo como un literal entero. Es decir, que el tamaño del arreglo no puede ser una expresión aritmética, el resultado de un método ni leerse de una variable. El tamaño de un arreglo no inicializado es cero.

Esta clase provee un único método que puede invocarse en las variables de esta clase, sin importar el tipo de sus elementos:

- `fn length() -> I32`
  length devuelve el tamaño del arreglo.

Al instanciar un arreglo, todos sus elementos se inicializan con el valor por defecto del tipo primitivo elegido.

```
fn main() {
  Array I32: numeros; // Declarar variable.
  (IO.out_array(numeros)); // Salida: []

  numeros = new I32[3]; // Asignar variable.
  (IO.out_array(numeros)); // Salida: [0, 0, 0]
}
```

El acceso a los elementos dentro de arreglo se hace a través de su índice, `a[i]` donde `a` es un arreglo e `i` es el índice que ubica un elemento del arreglo. Los índices de los arreglos comienzan en cero. A la hora de acceder un elemento está permitido utilizar todo tipo de expresiones de tipo I32, como variables o expresiones aritméticas.

```
fn main() {
  Array I32: numeros;
  numeros = new I32[3];

  numeros[1] = 5; // Asignar elemento
  (IO.out_i32(numeros[1])); // Salida: 5

  numeros[numeros.length() - 1] = 10 // Asignar ultimo elemento
  (IO.out_i32(numeros[2])); // Salida: 10
}
```

Si se intenta acceder a un índice fuera del rango del arreglo, se lanzará un error en tiempo de ejecución. El rango de índices válidos de un arreglo es `0 < i < a.length()`. Nótese que si el arreglo es de tamaño 10, los índices van del 0 al 9, por lo cual ocurrirá un error si se intenta acceder al índice 10.

```
fn main() {
  Array Str: cadenas;
  cadenas = new Str[3];
  cadenas[3] = "hola"; // ERROR acceso fuera de indice.
  cadenas[10] = "mundo"; // ERROR acceso fuera de indice.
}
```

Object

La clase Object es la superclase de todas las clases de TinyRust+, no posee métodos ni atributos. No está permitido declarar otra clase con el mismo nombre, pero a diferencia de las demás clases predefinidas, sí se puede heredar explícitamente de esta clase. Esto se debe a que todas las clases que no hereden explícitamente de otra clase heredan implícitamente de Object. Por lo cual hacer esta herencia explícita no representa ninguna funcionalidad extra. Es posible declarar una variable de tipo Object, pero no poseerá ningún atributo ni método accesible.

IO

La clase IO provee métodos estáticos útiles para realizar entrada/salida. A continuación se listan los métodos que provee.

Los siguientes métodos imprimen por pantalla (Standard Output[4]) el valor con el tipo dado en su único argumento. No retornan ningún valor dentro de TinyRust+. En caso de invocarse con un valor del tipo equivocado, el método arrojará un error en tiempo de ejecución.

- `static fn out_str(Str: s) -> void`
- `static fn out_i32(I32: i) -> void`
- `static fn out_bool(Bool: b) -> void`
- `static fn out_char(Char: c) -> void`
- `static fn out_array(Array: a) -> void`

En las secciones previas de este manual ya se pudo adelantar su forma de uso, en el siguiente fragmento de código se muestran más ejemplos básicos de uso de este método.

```
fn main() {
```

[4] Standard streams

```
  (IO.out_i32(10)); // Salida: 10
  (IO.out_str("Hola mundo!")); // Salida: Hola mundo!
  (IO.out_char('c')); // Salida: c
  (IO.out_bool(true)); // Salida: true
  (IO.out_array(new I32[3])); // Salida: [0, 0, 0]
}
```

Los métodos a continuación son la contracara de los anteriores: no reciben ningún argumento, pero retornan un valor del tipo dado leído desde el medio de entrada (Standard Input[4]), especificado de forma interactiva por el usuario en tiempo de ejecución.

- `static fn in_str() -> Str`
- `static fn in_i32() -> I32`
- `static fn in_bool() -> Bool`
- `static fn in_char() -> Char`

Todos estos métodos son estáticos, por lo cual no es necesario inicializar una instancia de la clase para accederlos. Está permitido heredar de la clase IO, a diferencia de la mayoría de clases predefinidas, ya que no representa ningún tipo de dato en particular, sino que provee funcionalidad. A pesar de esto, ninguno de sus métodos se puede redefinir, dado que son todos estáticos. A su vez, se pueden declarar variables e instanciar objetos de tipo IO, pero no provee ninguna funcionalidad extra, ya que los únicos métodos que provee son estáticos, de modo que igualmente se pueden acceder sin instanciar la clase.

Literales nulos

En TinyRust+ no existe una clase que represente a los literales nulos, pero esto no implica que no existan. El literal `nil` representa un valor nulo. Para indicar que un método retorna un valor de este tipo o que no retorna ningún valor (lo cual es equivalente semánticamente), se utiliza la palabra *void*. Se podría decir de manera informal que el tipo del literal `nil` es *void*. En el siguiente fragmento de código se ilustran las formas de retorno válido para un método que retorna un valor de tipo *void*.

```
fn out_str(Str: s) -> void { }

fn out_str(Str: s) -> void {
  return;
}

fn out_str(Str: s) -> void {
  return nil;
```

```
}
```

Clases

Como ya se adelantó, un archivo de TinyRust+ consiste en una serie de declaraciones de clases, seguido de la declaración del método *main*. El código en TinyRust+ se organiza en clases y se pueden definir múltiples clases en un mismo archivo. Todas las clases tienen visibilidad global, es decir que serán visibles entre sí y para el método *main*. La definición de una clase tiene la siguiente forma:

⟨Clase⟩ ::= **`class idClase`** ⟨Herencia⟩**?** **`{`** ⟨Miembro⟩***** **`}`**

⟨Herencia⟩ ::= **`:`** ⟨Tipo⟩

⟨Miembro⟩ ::= ⟨Atributo⟩ | ⟨Método⟩ | ⟨Constructor⟩

Los identificadores de clase deben comenzar con una letra mayúscula y no deben coincidir con el nombre de ninguna clase predefinida del lenguaje ni con el nombre de otra clase definida previamente en el archivo. Opcionalmente se puede especificar una clase de la cual hereda esta clase. Este lenguaje no soporta herencia múltiple. Los métodos y atributos públicos de la superclase serán accesibles dentro de la subclase como propios, a excepción de los métodos que sean redefinidos en la subclase, que quedarán inaccesibles.

```
class Figura { ... }

class Cuadrado: Figura { ... }

fn main() { ... }
```

El cuerpo de una definición de clase consiste en una lista de definiciones de miembros. Un miembro es un atributo o un método (posiblemente el constructor). Un atributo de una clase A especifica una variable que es parte del estado de objetos de la clase A. Un método de una clase A es un procedimiento que puede manipular variables y objetos de clase A.

Los nombres o identificadores de los miembros deben comenzar con letra minúscula. Dentro de una clase no se pueden definir métodos con el mismo nombre ni atributos con el mismo nombre. Pero un método y un atributo sí pueden tener el mismo nombre.

No hay ningún mecanismo en TinyRust+ para que los programadores desaloquen objetos. TinyRust+ tiene gestión automática de memoria.

Atributos

Un atributo especifica una variable que es parte del estado de objetos de la clase. La definición de un atributo de clase tiene la siguiente forma:

⟨Atributo⟩ ::= **`pub`**? ⟨Tipo⟩**`: id;`**

También es posible declarar una lista de atributos del mismo tipo en una sola línea:

⟨Atributo⟩ ::= **pub?** ⟨Tipo⟩**:** ⟨Lista-Declaración-Variables⟩**;**

Dentro de una misma clase no pueden declararse dos atributos con el mismo nombre. Los identificadores de atributos comienzan con una letra minúscula. Una restricción extra es que no se puede declarar ningún atributo con el identificador "self", dado que éste es tratado de forma especial y referencia a la clase misma.

Los atributos no pueden ser redefinidos en una subclase. Todos los atributos tienen alcance privado, salvo aquellos que tengan la palabra reservada *pub* en su declaración. La única forma de proveer acceso a los atributos no públicos de una clase es a través de sus métodos. Los atributos públicos son visibles tanto en subclases como en otras clases y el ámbito global.

Un comportamiento interesante es que los atributos privados de otras instancias de la misma clase son accesibles. Este comportamiento se ilustra en el siguiente ejemplo:

```
class Cuadrado {
  I32: lado;

  /** Retorna true si este cuadrado es mas grande que el
cuadrado provisto como argumento. Devuelve false en otro caso */
  fn comparar(Cuadrado: c) -> Bool {
    return self.lado > c.lado; //Acceder atributo privado c.lado
  }
}
```

En el ejemplo la clase `Cuadrado` posee un atributo privado `lado`. A su vez la clase implementa el método `comparar` que recibe otra instancia de esta clase como argumento y compara el valor del atributo lado de ambas instancias. Si se intentara acceder al atributo privado `lado` de la variable `c` de clase `Cuadrado` en cualquier otro contexto, se obtendrá un error en tiempo de compilación, dado que el atributo es inaccesible. La única excepción a esta regla es cuando el atributo privado se accede dentro de un método de la misma clase, sin importar que probablemente se acceda desde otra instancia de la clase en tiempo de ejecución.

Métodos

Un método de una clase A es un procedimiento que puede manipular variables y objetos de clase A. La definición de un método tiene la siguiente forma:

⟨Método⟩ ::= **static? fn id (** ⟨Lista-Argumentos-Formales⟩**?) ->**
⟨Tipo-Método⟩ ⟨Bloque-Método⟩

⟨Lista-Argumentos-Formales⟩ ::= ⟨Tipo⟩**: id,** ⟨Lista-Argumentos-Formales⟩

| ⟨Tipo⟩: **id**

⟨Tipo-Método⟩ ::= ⟨Tipo⟩ | **void**

⟨Bloque-Método⟩ ::= **{** ⟨Decl-Var-Locales⟩* ⟨Sentencia⟩* **}**

Los identificadores de métodos comienzan con una letra minúscula. Una restricción extra es que no se puede declarar ningún método con el identificador “create”, dado que éste es tratado de forma especial como el constructor de la clase. El constructor tiene una sintaxis ligeramente distinta al del resto de métodos, por lo cual el método *create* se debe declarar con tal sintaxis.

Todos los métodos tienen alcance global. Aquellos métodos que posean al comienzo de su declaración la palabra clave static serán métodos de clase, mientras que todos los demás métodos son de instancia. Dentro de métodos estáticos no pueden accederse atributos ni métodos de instancia, dado que es un contexto estático. Tampoco se puede acceder a la referencia *self*, ni siquiera para invocar métodos estáticos de la misma clase.

Cada método puede tener cero o más parámetros formales. Los identificadores utilizados en la lista de parámetros formales deben ser todos distintos entre sí. El tipo del cuerpo del método debe ajustarse al tipo de retorno declarado. Cuando se invoca un método, los parámetros formales se vinculan a los argumentos reales y se evalúa la expresión; el valor resultante es el significado de la invocación del método. Un parámetro formal oculta cualquier definición de un atributo del mismo nombre.

```
class Cuadrado {
  I32: lado;

  fn imprimir(Str: lado) -> void {
    (IO.out_str(lado)); // Accede al argumento Str.
    (IO.out_i32(self.lado)); // Accede al atributo I32.
  }
}
```

En caso de que un método no devuelva ningún valor o retorne un literal nulo, el tipo de retorno es *void*. Si el método tiene un tipo de retorno definido, entonces todas las ramas de ejecución condicional del bloque del método deben retornar un valor de este tipo. No respetar esta regla generará un error en tiempo de compilación. Es fácil ilustrar este concepto con un ejemplo:

```
fn mayor(I32: x, I32: y) -> Bool {
  if (x > y) {
    return true;
  }
}
```

En el ejemplo se retorna un valor de tipo booleano en caso de que la variable `x` tenga un valor mayor que la variable `y`. Pero en caso de que la condición no se cumpla, el método retornará un valor de tipo *void*, lo cual no coincide con la firma del método.

En una subclase está permitido redefinir métodos no estáticos de la superclase, pero para garantizar la seguridad de los tipos, existen restricciones sobre la redefinición de métodos heredados.

Si una clase *B* hereda un método *f* de una superclase *A,* entonces *B* puede anular la definición heredada de *f* siempre que el número de argumentos, los tipos de argumentos formales y el tipo de retorno sean exactamente iguales en ambas definiciones. Un método de clase no se puede redefinir, ni se puede redefinir un método de instancia como método de clase. Para ver por qué es necesaria alguna restricción en la redefinición de métodos heredados, se puede considerar el siguiente ejemplo:

```
class A {
  fn f() -> I32 { return 1; }
}

class B: A {
  fn f() -> Str { return "1"; }
};
```

En el ejemplo previo, si se tomara un elemento `a` con tipo dinámico `A`, entonces `a.f() + 1` sería una expresión bien formada con valor 2. Sin embargo, no sería posible sustituir un valor de tipo `B` para `a`, ya que se trataría de sumar un número con una cadena. Luego, si los métodos pudieran redefinirse de forma arbitraria, entonces las subclases no podrían extender de forma sencilla el comportamiento de sus superclases y provocarían, posiblemente, errores de tipo en tiempo de ejecución.

Constructor

Cada clase puede contener opcionalmente la declaración de un único constructor. El constructor es un método cuya definición es llamada únicamente cuando un nuevo objeto de la clase es inicializado. El nombre de este método es *create* y su declaración tiene la siguiente forma:

⟨Constructor⟩ ::= **`create`** **`(`**⟨Lista-Argumentos-Formales⟩**`?`**`)` ⟨Bloque-Método⟩

El método *create* es el único método que puede tener argumentos distintos en una subclase y una superclase. Este método únicamente es invocado al instanciar la clase, pero no puede ser accedido directamente como cualquier otro método.

```
class A {
  I32: x;
  create(I32: x) {
```

```
    // La inicializacion de la clase se coloca aqui.
    self.x = x;
  }
}
fn main() {
  A: varA;
  varA = new A(10); // Invoca el metodo create.
}
```

Herencia

Como ya se adelantó, la herencia de clases está permitida en TinyRust+. La herencia se define de la siguiente manera:

```
class B: A {
  ...
}
```

Si se declara una clase B que hereda de una clase A entonces B hereda los miembros de A. Se dice que la clase A es la superclase de B y B es la subclase de A. La semántica de B hereda de A es que B tiene todos los miembros definidos en A además de los suyos propios.

En el caso de que una superclase y una subclase definan el mismo nombre de método, entonces la definición dada en la subclase tiene prioridad.

Es ilegal redefinir los nombres de los atributos.

Además, para la seguridad de los tipos, se imponen algunas restricciones sobre cómo se pueden redefinir los métodos. Si una definición de clase no especifica una clase principal, entonces la clase hereda de Object de forma predeterminada. Una clase puede heredar sólo de una sola clase (no se permite herencia múltiple). La relación superclase-subclase en las clases define un grafo. Este grafo no puede contener ciclos. Por ejemplo, si B hereda de A, entonces A no debe heredar de B. Además, si B hereda de A, entonces A debe tener una definición de clase en algún lugar del código fuente. Debido a que TinyRust+ tiene una herencia única, se deduce que si se satisfacen ambas restricciones, el grafo de herencia forma un árbol con la clase *Object* como raíz.

Se permite heredar de las clases predefinidas *IO* y *Object*, pero intentar extender de cualquier otra clase predefinida generará un error en tiempo de compilación. TinyRust+ valida en tiempo de compilación que todas las clases de las cuales se hereda estén definidas (o sean predefinidas) y que no exista herencia cíclica. En este proceso se valida que ninguna clase herede directa o indirectamente de sí misma. Por ejemplo, el siguiente archivo generaría un error de compilación debido a una herencia cíclica en la clase `Cuadrado`, ya que se formaría un bucle de herencia entre las clases `Cuadrado` -> `Cuadrilatero` -> `Figura` -> `Cuadrado`.

```
class Figura: Cuadrado { ... }

class Cuadrilatero: Figura { ... }

class Rectangulo: Cuadrilatero { ... }

class Cuadrado: Cuadrilatero { ... }

fn main() { ... }
```

Tipos

En TinyRust+, toda clase también es un tipo. Una declaración de tipos tiene la forma `A: x`, donde `x` es una variable de tipo A. Toda variable debe tener una declaración de tipo en el punto en que aparece, por ejemplo, como parámetro formal en un método. Los tipos de todos los atributos deben estar también declarados.

La regla de tipo básica en TinyRust+ es que si un método o variable espera un valor de tipo A, entonces cualquier valor del tipo B se puede utilizar en su lugar, siempre que A sea un antepasado de B en la jerarquía de clases.

El sistema de tipos garantiza en tiempo de compilación que la ejecución de un programa no resulte en un error de tipo en tiempo de ejecución. Utilizando los tipos declarados para los identificadores en el código fuente, durante el análisis se infiere un tipo para toda expresión en el código fuente.

Es importante distinguir entre el tipo asignado por el verificador de tipos para una expresión en tiempo de compilación (tipo estático de la expresión) y los tipos o tipo que la expresión puede evaluar durante la ejecución (tipos dinámicos).

La distinción entre tipo estático y tipo dinámico es necesaria debido a que el verificador de tipos no puede, en tiempo de compilación, tener información perfecta sobre qué valor se va a computar en tiempo de ejecución. De hecho, el tipo estático y el tipo dinámico de las expresiones suelen ser distintos. Lo que es necesario, sin embargo, es que los tipos estáticos del verificador de tipos sean correctos con respecto a los tipos dinámicos. De este modo, se garantiza en tiempo de compilación que los métodos y atributos accedidos en la ejecución están definidos, pero en tiempo de compilación no se tiene certeza de cuál es el cuerpo de método que finalmente se ejecutará en una invocación de método.

Sentencias

Las sentencias en TinyRust+ incluyen a las expresiones, asignaciones, declaraciones ya sea de constantes, atributos y métodos, bucles, bloques y condicionales. En las siguientes secciones se explica con detalle cada tipo de sentencia.

Vale destacar que todas las sentencias que no terminen con un bloque deben obligatoriamente terminar con un punto y coma. Además de esto, el punto y coma es una sentencia en sí misma, por lo cual puede igualmente agregarse al final de un bloque o repetirse múltiples veces al final de una misma línea.

```
if (cond) {
  return true;;;;
};;
```

Asignación

La sentencia de asignación permite asociar un valor a una variable. La expresión asignada debe ser del mismo tipo estático que la variable a la cual se va a asignar. La sintaxis es la siguiente:

⟨Asignación⟩ ::= ⟨Acceso-Var-Simple⟩ **=** ⟨Expresión⟩**;**
| ⟨Acceso-Self-Simple⟩ **=** ⟨Expresión⟩**;**

⟨Acceso-Var-Simple⟩ ::= **id** ⟨Encadenado-Simple⟩* | **id [** ⟨Expresión⟩ **]**

⟨Acceso-Self-Simple⟩ ::= **self** ⟨Encadenado-Simple⟩*

⟨Encadenado-Simple⟩ ::= **.id**

La sintaxis deja ver que no sólo se pueden asignar variables locales, sino también atributos públicos de otras variables y atributos de la clase actual también. Una restricción es que no se puede asignar un valor a *self*, dado que es una referencia a la clase actual, no es una variable tradicional. En el siguiente fragmento de código, todas las asignaciones dentro del método *f* de la clase *A* son válidas en TinyRust+.

```
class B {
  pub A: x;
}

class A {
  pub I32: y;

  fn f(B: b) -> void {
    Array Bool: arreglo;
```

```
    y = 1; // Atributo
    self.y = 2; // Atributo
    b.x.y = 3; // Atributo publico
    arreglo[1] = true; // Elemento de un arreglo
  }
}
```

Sentencia simple

TinyRust+ realiza llamadas a métodos y utiliza otras expresiones como una sentencia de la siguiente forma:

⟨Sentencia-Simple⟩ ::= **(** ⟨Expresión⟩ **);**

Esta sintaxis permite utilizar expresiones de cualquier tipo en un rol de sentencias. Ejemplos de esto ya se han visto previamente al invocar métodos de la clase IO:

```
fn main() {
  (IO.out_str("hola mundo"));
}
```

Bloque

Un bloque en TinyRust+ respeta la siguiente forma:

⟨Bloque⟩ ::= **{** ⟨Sentencia⟩* **}**

Las sentencias dentro del bloque se evalúan en orden de izquierda a derecha. Los bloques en TinyRust+ se diferencian entre bloque de método (aquellos que contienen declaraciones de variables) y bloque de sentencias (bloques dentro de if-else, while, etc. que no tienen declaraciones de variables). Esta distinción se puede ver claramente en la gramática del lenguaje en el Anexo 2.

Condicional

Los condicionales permiten ejecutar código bajo ciertas condiciones. Si bien durante la ejecución pueden no ejecutarse una rama del condicional, el código de todas las ramas es analizado en tiempo de compilación. La sintaxis del condicional es la siguiente:

⟨Condicional⟩ ::= **if (**⟨Expresión⟩**)** ⟨Sentencia⟩
| **if (**⟨Expresión⟩**)** ⟨Sentencia⟩ **else** ⟨Sentencia⟩

La semántica de los condicionales es la utilizada normalmente. Primero se evalúa la condición, que debe ser una expresión de tipo booleana. Si es verdadera, se evalúa el código que está dentro del *if*. Especificar un else es opcional, si la condición es falsa y se definió una rama *else*, se evalúa el código dentro del *else*.

El lenguaje no implementa otro tipo de sentencias condicionales como *switch* u operadores ternarios, pero su comportamiento se puede lograr fácilmente haciendo uso de la sentencia *if-else*.

```
fn main() {
  I32: valor;
  Bool: condicion;

  // Ejemplo operador ternario: valor = condicion ? 10 : 5;
  if (condicion) valor = 10;
  else valor = 5;

  // Ejemplo switch case:
  if (valor == 0) {
    (IO.out_str("case 0"));
  else if (valor == 5) {
    (IO.out_str("case 5"));
  else if (valor == 10) {
    (IO.out_str("case 10"));
  } else {
    (IO.out_str("default case"));
  }
}
```

Bucle

TinyRust+ únicamente implementa bucles de tipo *while*. Igualmente el comportamiento de otros tipos de bucle, como *for* o *do while* se puede simular mediante el bucle *while*. La sintaxis para un bucle *while* en TinyRust+ respeta la siguiente forma:

⟨Bucle⟩ ::= **`while (`**⟨Expresión⟩**`)`** ⟨Sentencia⟩

La condición debe ser una expresión booleana y se evalúa antes de cada iteración del ciclo. Si la condición es falsa, la ejecución del bucle termina. Si la condición es verdadera, se evalúa el cuerpo del bucle y el proceso se repite.

Vale destacar que TinyRust+ no implementa sentencias de tipo *break* o *continue*. El comportamiento dado por estas sentencias se puede lograr haciendo uso de condicionales dentro del bucle.

En el siguiente fragmento de código se provee una implementación del comportamiento de un bucle *for* utilizando el bucle *while* de TinyRust+.

```
fn main() {
```

```
  I32: i, max, step;
  i = 0; max = 10; step = 1;

  while (i < max) {
    (IO.out_i32(i+1)); // Imprime los numeros del 1 al 10.
    i = i + step;
  }
}
```

Retorno de método

La sentencia de retorno se utiliza dentro de métodos y determina el valor de retorno del método. Esta sentencia tiene la siguiente forma:

⟨Retorno⟩ ::= **`return`** ⟨Expresión⟩`?;`

El tipo de la expresión debe coincidir con el tipo de retorno del método. Es obligatorio su uso dentro de métodos que retornan un valor de un tipo distinto de void. Si el tipo de retorno es void, entonces su uso es opcional y puede utilizarse sin incluir ninguna expresión en la sentencia.

```
class A {
  fn a() -> void {
    if (1 > 2) {
      return;
    }
  }

  fn b() -> A {
    return new A();
  }
}
```

Vale destacar que TinyRust+ no soporta el retorno múltiple, es decir que en este lenguaje se puede retornar a lo sumo un único valor en una sentencia de retorno.

Expresiones

Una expresión es un fragmento de código que produce un valor. En TinyRust+ son expresiones las operaciones de todo tipo, llamadas a métodos, accesos a atributos, accesos a variables, la referencia *self* e instanciación de clases.

Operaciones

TinyRust+ tiene cinco operaciones aritméticas binarias: `+`, `-`, `*`, `/` y `%`.

Su sintaxis es:

⟨OperacionBinaria⟩ ::= ⟨Expresión⟩ ⟨Operador⟩ ⟨Expresión⟩

Para evaluar la expresión binaria, se evalúa la primera expresión y luego la segunda. El resultado de la operación es el resultado de la expresión. Los tipos estáticos de los dos operandos debe ser I32. El tipo estático de la expresión completa es I32. Vale destacar que la división también retorna un entero, dado que se realiza una división entera. Si se intentara realizar una división por cero, ocurriría un error en tiempo de ejecución.

Se pueden formar expresiones booleanas utilizando los operadores lógicos `&&` (and), `||`(or) y `!` (not). También se pueden generar expresiones de este tipo utilizando los operadores relacionales sobre enteros: `<`, `<=`, `==`, `>=` y `!=` .

La comparación por igual `==` es un caso especial. Si alguna de las dos expresiones tiene el tipo estático I32, Char, Bool o Str, el otro debe tener el mismo tipo estático y el mismo valor. Cualquier otro tipo puede compararse libremente. En objetos no básicos, la igualdad simplemente verifica la igualdad de la referencia (es decir, si las direcciones de memoria de los objetos son las mismas).

Llamada a métodos

Las llamadas a un método dinámico en TinyRust+ suelen tener la siguiente forma:

⟨Llamada-Método⟩ ::= ⟨Expresión⟩ **`. id (`**⟨Lista-Expresiones⟩**`?)`** ⟨Encadenado⟩**`?`**

⟨Lista-Expresiones⟩ ::= ⟨Expresión⟩ | ⟨Expresión⟩ **`,`** ⟨Lista-Expresiones⟩

En caso de estar dentro de una clase, se pueden invocar métodos de la misma clase simplemente por su nombre:

⟨Llamada-Método⟩ ::= **`id (`**⟨Lista-Expresiones⟩**`?)`** ⟨Encadenado⟩**`?`**

Para acceder métodos estáticos de una clase, se utiliza la siguiente sintaxis:

⟨Llamada-Método-Estático⟩ ::= **`idClase.id(`**⟨Lista-Expresiones⟩**`?)`** ⟨Encadenado⟩**`?`**

Donde *idClase* es el nombre de la clase e *id* es el nombre del método estático. Cabe

destacar que igualmente los métodos estáticos de una clase se pueden acceder a través de una instancia de la clase utilizando la misma sintaxis de invocación de los métodos dinámicos.

En TinyRust+ los métodos siempre se llaman con los paréntesis aunque no tengan parámetros. El tipo de esta expresión es igual al tipo de retorno del método. Al acceso a método pueden encadenarse accesos a otros métodos y atributos públicos de la clase retornada. En este caso, el valor final de la expresión completa será el del último valor de la cadena de accesos encadenados.

```
class A {
  pub B: b;
  fn mA() -> A { return self; }
}

class B {
  pub A: a;
  fn mB() -> B { return self; }

  fn f() -> void {
    (mB()); // Accesible. Tipo: B
    (a.mA()); // Accesible. Tipo: A
    (mB().a.mA().mA().b.f()); // Accesible. Tipo: void
  }
}
```

Acceso a atributos

En TinyRust+ el acceso a atributos públicos de una clase se realiza con la siguiente sintaxis:

⟨Acceso-Var⟩ ::= ⟨Expresión⟩ **. id** ⟨Encadenado⟩**?**

Para acceder atributos públicos y privados dentro de la misma clase se pueden acceder directamente usando el nombre del atributo (siempre y cuando ninguna variable local lo esconda):

⟨Acceso-Var⟩ ::= **id** ⟨Encadenado⟩**?**

Si el atributo es de tipo arreglo, se puede especificar el índice del arreglo al cuál se quiere acceder. El tipo resultante de la expresión se corresponde con el tipo de los elementos del arreglo.

⟨Acceso-Var-Array⟩ ::= **id [** ⟨Expresión⟩ **]**

Únicamente se puede acceder al valor de atributos privados de una clase fuera de la

misma a través de los métodos que tenga definida dicha clase.

El valor de la expresión es igual al valor del atributo accedido. Al acceso al atributo pueden encadenarse accesos a otros métodos y atributos públicos del tipo del atributo original. En este caso, el valor final de la expresión completa será el del último valor de la cadena de accesos encadenados.

```
class A {
  I32: x;
  pub A: attrA;
  fn a() -> A {
    (x); // Accesible. Tipo: I32
    (new A().x); // Accesible. Tipo: I32
    (attrA.attrA.attrA.a().x); // Accesible. Tipo: I32

    return new A();
  }
}
```

New

Una expresión *new* retorna una nueva instancia de la clase dada. Su sintaxis es:

⟨Llamada-Constructor⟩ ::= **`new idClase`** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩?

El valor es un objeto nuevo de la clase apropiada. A la expresión new pueden encadenarse accesos a métodos y atributos públicos de la clase. En este caso, el valor final de la expresión completa será el del último valor de la cadena de accesos encadenados.

Para el caso de los arreglos la sintaxis de inicialización es:

⟨Llamada-Constructor-Array⟩ ::= **`new`** ⟨Tipo-Primitivo⟩ **`[`** ⟨Literal-Entero⟩ **`]`**

La invocación a *new* realiza la reserva de espacio para la referencia al objeto y la inicialización de la clase. Se llama al método *create* de la clase si es que existe. Cuando se instancia un nuevo objeto de una clase, se deben inicializar todos los atributos heredados (públicos) y locales. Los atributos heredados se inicializan primero en orden de herencia comenzando con los atributos de la clase de ancestro más grande. Dentro de una clase determinada, los atributos se inicializan en el orden en que aparecen en el código fuente.

La invocación a *new* para un Array implica que se reserva en memoria el tamaño especificado por el valor del literal entero y sus elementos son inicializados con el valor por defecto según el tipo primitivo elegido. Además, el tipo declarado para los elementos debe coincidir con el tipo de la sentencia *new*.

```
fn main() {
  Array I32: a;
```

```
  a = new I32[6]; // Reserva el espacio de 6 elementos enteros y
los inicializa a cada uno con cero.
  (IO.out_i32(a[2]); // Imprime 0, el valor por defecto.
}
```

Self

El identificador self es una referencia explícita a la instancia del tipo en el que ocurre. Dado que retorna un valor, se puede considerar una expresión.

Anexo 1: alfabeto de entrada

El alfabeto de entrada de TinyRust+ consta de los caracteres con código ASCII desde 32 hasta 126[5]. Adicionalmente, también se incluyen los caracteres:

- Salto de línea (ASCII 10)
- Tabulación (ASCII 9)
- Fin de texto (ASCII 3)
- Tabulación vertical (ASCII 11)

En la siguiente tabla se encuentra definido por extensión el alfabeto de TinyRust+.

[space]	!	"	#	$	%	&	'	(	)	*	+	,	-	.	0	1	2	3	4
[tab]	5	6	7	8	9	:	;	<	=	>	?	@	A	B	C	D	E	F	G
[vtab]	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
[break]	[	]	_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
[etx]	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	\	/	^	

Cabe destacar que ciertos caracteres exclusivos del español, como “ñ” y vocales con tilde (tanto mayúsculas como minúsculas) no forman parte del alfabeto de entrada de este compilador.

[5] ASCII Code. ASCII table according to Windows-1252

Anexo 2: gramática EBNF

A continuación se especifica la gramática libre de contexto en formato BNF extendido del lenguaje TinyRust+. En letra **mecanográfica** y **negrita** aparecen los tokens devueltos por el Analizador Léxico. Los símbolos no terminales de la gramática se escriben dentro de ⟨símbolos⟩. La notación propia de EBNF se encuentra resaltada en color anaranjado para no confundirse con los tokens del lenguaje.

⟨program⟩ ::= ⟨Clase⟩* ⟨Método-Main⟩

⟨Método-Main⟩ ::= **`fn main ()`** ⟨Bloque-Método⟩

⟨Clase⟩ ::= **`class id`** ⟨Herencia⟩? **`{`** ⟨Miembro⟩* **`}`**

⟨Herencia⟩ ::= : ⟨Tipo⟩

⟨Miembro⟩ ::= ⟨Atributo⟩ | ⟨Método⟩ | ⟨Constructor⟩

⟨Constructor ⟩ ::= **`create`** ⟨Argumentos-Formales⟩ ⟨Bloque-Método⟩

⟨Atributo⟩ ::= ⟨Visibilidad ⟩? ⟨Tipo⟩**`:`** ⟨Lista-Declaración-Variables⟩**`;`**

⟨Método⟩ ::= ⟨Forma-Método⟩? **`fn id`** ⟨Argumentos-Formales⟩ **`->`** ⟨Tipo-Método⟩ ⟨Bloque-Método⟩

⟨Visibilidad ⟩ ::= **`pub`**

⟨Forma-Método⟩ ::= **`static`**

⟨Bloque-Método⟩ ::= **`{`** ⟨Decl-Var-Locales⟩* ⟨Sentencia⟩* **`}`**

⟨Decl-Var-Locales⟩ ::= ⟨Tipo⟩**`:`** ⟨Lista-Declaración-Variables⟩**`;`**

⟨Lista-Declaración-Variables⟩::= **`id`** | **`id ,`** ⟨Lista-Declaración-Variables⟩

⟨Argumentos-Formales⟩::= **`(`** ⟨Lista-Argumentos-Formales⟩? **`)`**

⟨Lista-Argumentos-Formales⟩ ::= ⟨Argumento-Formal ⟩**`,`** ⟨Lista-Argumentos-Formales⟩ | ⟨Argumento-Formal ⟩

⟨Argumento-Formal ⟩ ::= ⟨Tipo⟩**`: id`**

⟨TipoMétodo⟩ ::= ⟨Tipo⟩ | **`void`**

⟨Tipo⟩ ::= ⟨Tipo-Primitivo⟩ | ⟨Tipo-Referencia⟩ | ⟨Tipo-Arreglo⟩

⟨Tipo-Primitivo⟩ ::= **`Str`** | **`Bool`** |**`I32`** | **`Char`**

⟨Tipo-Referencia⟩ ::= **`id`**

⟨Tipo-Arreglo⟩ ::= **`Array`** ⟨Tipo-Primitivo⟩

⟨Sentencia⟩ ::= **`;`**
| ⟨Asignación⟩**`;`**
| ⟨Sentencia-Simple⟩**`;`**
| **`if (`**⟨Expresión⟩**`)`** ⟨Sentencia⟩
| **`if (`**⟨Expresión⟩**`)`** ⟨Sentencia⟩ **`else`** ⟨Sentencia⟩
| **`while (`** ⟨Expresión⟩ **`)`** ⟨Sentencia⟩
| ⟨Bloque⟩
| **`return`** ⟨Expresión⟩**`?;`**

⟨Bloque⟩ ::= **`{`** ⟨Sentencia⟩* **`}`**

⟨Asignación⟩ ::= ⟨Acceso-Var-Simple⟩ **`=`** ⟨Expresión⟩
| ⟨Acceso-Self-Simple⟩ **`=`** ⟨Expresión⟩

⟨Acceso-Var-Simple⟩ ::= **`id`** ⟨Encadenado-Simple⟩* | **`id [`** ⟨Expresión⟩ **`]`**

⟨Acceso-Self-Simple⟩ ::= **`self`** ⟨Encadenado-Simple⟩*

⟨Encadenado-Simple⟩ ::= **`.id`**

⟨Sentencia-Simple⟩ ::= **`(`** ⟨Expresión⟩ **`)`**

⟨Expresión⟩ ::= ⟨ExpOr ⟩

⟨ExpOr ⟩ ::= ⟨ExpOr ⟩ **`||`** ⟨ExpAnd ⟩ | ⟨ExpAnd ⟩

⟨ExpAnd ⟩ ::= ⟨ExpAnd ⟩ **`&&`** ⟨ExpIgual ⟩ | ⟨ExpIgual ⟩

⟨ExpIgual ⟩ ::= ⟨ExpIgual ⟩⟨OpIgual ⟩⟨ExpCompuesta⟩ | ⟨ExpCompuesta⟩

⟨ExpCompuesta⟩ ::= ⟨ExpAd ⟩⟨OpCompuesto⟩⟨ExpAd ⟩ | ⟨ExpAd ⟩

⟨ExpAd ⟩ ::= ⟨ExpAd ⟩⟨OpAd ⟩⟨ExpMul ⟩ | ⟨ExpMul ⟩

⟨ExpMul ⟩ ::= ⟨ExpMul ⟩⟨OpMul ⟩⟨ExpUn⟩ | ⟨ExpUn⟩

⟨ExpUn⟩ ::= ⟨OpUnario⟩⟨ExpUn⟩ | ⟨Operando⟩

⟨OpIgual ⟩ ::= **`==`** | **`!=`**

⟨OpCompuesto⟩ ::= **`<`** | **`>`** | **`<=`** | **`>=`**

⟨OpAd ⟩ ::= **`+`** | **`-`**

⟨OpUnario⟩ ::= **`+`** | **`-`** | **`!`**

⟨OpMul ⟩ ::= **`*`** | **`/`** | **`%`**

⟨Operando⟩ ::= ⟨Literal ⟩ | ⟨Primario⟩ ⟨Encadenado⟩**?**

⟨Literal ⟩ ::= **nil** | **true** | **false** | **intLiteral** | **StrLiteral**
| **charLiteral**

⟨Primario⟩ ::= ⟨Expresión-Parentizada⟩
| ⟨Acceso-Self⟩
| ⟨Acceso-Var⟩
| ⟨Llamada-Método⟩
| ⟨Llamada-Método-Estático⟩
| ⟨Llamada-Constructor⟩

⟨Expresión-Parentizada⟩ ::= **(** ⟨Expresión⟩ **)** ⟨Encadenado⟩**?**

⟨Acceso-Self ⟩ ::= **self** ⟨Encadenado⟩**?**

⟨Acceso-Var ⟩ ::= **id** ⟨Encadenado⟩**?** | **id [** ⟨Expresión⟩ **]**

⟨Llamada-Método⟩ ::= **id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩**?**

⟨Llamada-Método-Estático⟩ ::= **id .** ⟨Llamada-Método⟩ ⟨Encadenado⟩**?**

⟨Llamada-Constructor ⟩ ::= **new id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩**?**
| **new** ⟨Tipo-Primitivo⟩ **[** ⟨Expresión⟩ **]**

⟨Argumentos-Actuales⟩ ::= **(** ⟨Lista-Expresiones⟩**?** **)**

⟨Lista-Expresiones⟩ ::= ⟨Expresión⟩ | ⟨Expresión⟩ **,** ⟨Lista-Expresiones⟩

⟨Encadenado⟩ ::=**.**⟨Llamada-Método-Encadenado⟩ | **.**⟨Acceso-Variable-Encadenado⟩

⟨Llamada-Método-Encadenado⟩ ::= **id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩**?**

⟨Acceso-Variable-Encadenado⟩ ::= **id** ⟨Encadenado⟩**?** | **id [** ⟨Expresión⟩ **]**